

TCG Storage Security Subsystem Class: Pyrite

**Specification Version 2.01
Revision 1.00**

May 7 2020

Contact: admin@trustedcomputinggroup.org

TCG

PUBLISHED

Copyright © TCG 2020

Copyright © 2020 Trusted Computing Group, Incorporated.

Disclaimers, Notices, and License Terms

THIS SPECIFICATION IS PROVIDED “AS IS” WITH NO WARRANTIES WHATSOEVER, INCLUDING ANY WARRANTY OF MERCHANTABILITY, NONINFRINGEMENT, FITNESS FOR ANY PARTICULAR PURPOSE, OR ANY WARRANTY OTHERWISE ARISING OUT OF ANY PROPOSAL, SPECIFICATION OR SAMPLE.

Without limitation, TCG disclaims all liability, including liability for infringement of any proprietary rights, relating to use of information in this specification and to the implementation of this specification, and TCG disclaims all liability for cost of procurement of substitute goods or services, lost profits, loss of use, loss of data or any incidental, consequential, direct, indirect, or special damages, whether under contract, tort, warranty or otherwise, arising in any way out of use or reliance upon this specification or any information herein.

This document is copyrighted by Trusted Computing Group (TCG), and no license, express or implied, is granted herein other than as follows: You may not copy or reproduce the document or distribute it to others without written permission from TCG, except that you may freely do so for the purposes of (a) examining or implementing TCG specifications or (b) developing, testing, or promoting information technology standards and best practices, so long as you distribute the document with these disclaimers, notices, and license terms.

Contact the Trusted Computing Group at www.trustedcomputinggroup.org for information on specification licensing through membership agreements.

Any marks and brands contained herein are the property of their respective owners.

TABLE OF CONTENTS

1	INTRODUCTION	1
1.1	DOCUMENT PURPOSE	1
1.2	SCOPE AND INTENDED AUDIENCE	1
1.3	KEY WORDS	1
1.4	DOCUMENT REFERENCES	1
1.5	DOCUMENT PRECEDENCE.....	1
1.6	SSC TERMINOLOGY	2
1.7	LEGEND	3
2	PYRITE SSC OVERVIEW	4
2.1	PYRITE SSC USE CASES AND THREATS.....	4
2.2	SECURITY PROVIDERS (SPs).....	4
2.3	INTERFACE COMMUNICATION PROTOCOL	4
2.4	AUTHENTICATION	5
2.5	TABLE MANAGEMENT.....	5
2.6	ACCESS CONTROL & PERSONALIZATION	5
2.7	ISSUANCE	5
2.8	SSC DISCOVERY	5
2.9	MBR SHADOWING	5
2.10	MANDATORY FEATURE SETS.....	5
3	PYRITE SSC FEATURES.....	6
3.1	SECURITY PROTOCOL 1 SUPPORT	6
3.1.1	<i>Level 0 Discovery (M)</i>	6
3.1.1.1	Level 0 Discovery Header	6
3.1.1.2	TPer Feature (Feature Code = 0x0001)	7
3.1.1.3	Locking Feature (Feature Code = 0x0002).....	7
3.1.1.3.1	MBR Done Definition	8
3.1.1.3.2	MBR Enabled Definition.....	8
3.1.1.3.3	Locking Enabled Definition	8
3.1.1.4	Pyrite SSC V2.00 Feature (Feature Code = 0x0303).....	9
3.1.1.5	Supported Data Removal Mechanism Feature (Feature Code = 0x0404)	10
3.1.1.5.1	Data Removal Operation Processing Definition.....	10
3.1.1.5.2	Supported Data Removal Mechanism Definition.....	10
3.1.1.5.3	Data Removal Time Format and Data Removal Time Definition	11
3.2	SECURITY PROTOCOL 2 SUPPORT	12
3.2.1	<i>ComID Management</i>	12
3.2.2	<i>Stack Protocol Reset (M)</i>	12
3.2.3	<i>TPER_RESET command (M)</i>	12
3.3	COMMUNICATIONS	14
3.3.1	<i>Communication Properties</i>	14
3.3.2	<i>Supported Security Protocols</i>	14
3.3.3	<i>ComIDs</i>	14
3.3.4	<i>Synchronous Protocol</i>	15
3.3.4.1	Payload Encoding.....	16
3.3.4.1.1	Stream Encoding Modifications	16
3.3.4.1.2	TCG Packets.....	16
3.3.4.1.3	Payload Error Response	16
3.3.5	<i>Storage Device Resets</i>	16
3.3.5.1	Interface Resets	16
3.3.5.2	TCG Reset Events	17
3.3.6	<i>Protocol Stack Reset Commands (M)</i>	17

4	PYRITE SSC-COMPLIANT FUNCTIONS AND SPS	18
4.1	SESSION MANAGER	18
4.1.1	Methods	18
4.1.1.1	Properties (M)	18
4.1.1.2	StartSession (M)	19
4.1.1.3	SyncSession (M)	19
4.1.1.4	CloseSession (O)	19
4.2	ADMIN SP	20
4.2.1	Base Template Tables	20
4.2.1.1	SPInfo (M)	20
4.2.1.2	SPTemplates (M)	20
4.2.1.3	Table (M)	21
4.2.1.4	MethodID (M)	22
4.2.1.5	AccessControl (M)	22
4.2.1.6	ACE (M)	29
4.2.1.7	Authority (M)	30
4.2.1.8	C_PIN (M)	30
4.2.2	Base Template Methods	31
4.2.3	Admin Template Tables	32
4.2.3.1	TPerInfo (M)	32
4.2.3.2	Template (M)	32
4.2.3.3	SP (M)	32
4.2.4	Admin Template Methods	33
4.2.5	Pyrite Additional Column Types	33
4.2.5.1	data_removal_mechanism	33
4.2.6	Pyrite Additional Data Structures	34
4.2.6.1	DataRemovalMechanism (Object Table)	34
4.2.6.1.1	UID	34
4.2.6.1.2	ActiveDataRemovalMechanism	34
4.2.7	Pyrite Additional Tables	34
4.2.7.1	DataRemovalMechanism (M)	34
4.2.8	Crypto Template Tables	35
4.2.9	Crypto Template Methods	35
4.2.9.1	Random	35
4.3	LOCKING SP	36
4.3.1	Base Template Tables	36
4.3.1.1	SPInfo (M)	36
4.3.1.2	SPTemplates (M)	36
4.3.1.3	Table (M)	36
4.3.1.4	Type (N)	37
4.3.1.5	MethodID (M)	38
4.3.1.6	AccessControl (M)	38
4.3.1.7	ACE (M)	50
4.3.1.8	Authority (M)	52
4.3.1.9	C_PIN (M)	53
4.3.2	Base Template Methods	53
4.3.3	Crypto Template Tables	54
4.3.4	Crypto Template Methods	54
4.3.4.1	Random	54
4.3.5	Locking Template Tables	55
4.3.5.1	LockingInfo (M)	55
4.3.5.2	Locking (M)	55
4.3.5.2.1	LockOnReset Restrictions	56
4.3.5.3	MBRControl (MBR O)	56
4.3.5.3.1	DoneOnReset Restrictions	56

4.3.5.4	MBR (MBR O).....	56
4.3.6	<i>Locking Template Methods</i>	57
4.3.7	<i>Non Template Tables</i>	57
4.3.7.1	DataStore (M).....	57
5	APPENDIX – SSC SPECIFIC FEATURES.....	58
5.1	PYRITE SSC-SPECIFIC METHODS.....	58
5.1.1	<i>Activate – Admin Template SP Object Method</i>	58
5.1.1.1	Activate Support.....	58
5.1.1.2	Side effects of Activate	58
5.1.2	<i>Revert – Admin Template SP Object Method</i>	59
5.1.2.1	Revert Support.....	59
5.1.2.2	Side effects of Revert	59
5.1.2.2.1	Effects of Revert on the PIN Column Value of C_PIN_SID.....	60
5.1.3	<i>RevertSP – Base Template SP Method</i>	60
5.1.3.1	RevertSP Support	60
5.1.3.2	KeepData parameter (Locking Template-specific).....	61
5.1.3.3	Side effects of RevertSP	61
5.2	LIFE CYCLE	62
5.2.1	<i>Issued vs. Manufactured SPs</i>	62
5.2.1.1	Issued SPs.....	62
5.2.1.2	Manufactured SPs.....	62
5.2.2	<i>Manufactured SP Life Cycle States</i>	62
5.2.2.1	State definitions for Manufactured SPs	62
5.2.2.2	State transitions for Manufactured SPs.....	63
5.2.2.2.1	Manufactured-Inactive to Manufactured	63
5.2.2.2.2	ANY STATE to ORIGINAL FACTORY STATE.....	63
5.2.2.3	State behaviors for Manufactured SPs.....	64
5.2.2.3.1	Manufactured-Inactive	64
5.2.2.3.2	Manufactured.....	64
5.2.3	<i>Type Table Modification</i>	64
5.3	BYTE TABLE ACCESS GRANULARITY	65
5.3.1	<i>Table Table Modification</i>	65
5.3.1.1	MandatoryWriteGranularity	65
5.3.1.1.1	Object Tables.....	65
5.3.1.1.2	Byte Tables.....	65
5.3.1.2	RecommendedAccessGranularity	66
5.3.1.2.1	Object Tables.....	66
5.3.1.2.2	Byte Tables.....	66

Tables

Table 1 Pyrite SSC Terminology	2
Table 2 SP Table Legend	3
Table 3 Level 0 Discovery Header	6
Table 4 Level 0 Discovery - TPer Feature Descriptor	7
Table 5 Level 0 Discovery - Locking Feature Descriptor	7
Table 6 Level 0 Discovery - Pyrite SSC Feature Descriptor	9
Table 7 Level 0 Discovery – Supported Data Removal Mechanism Feature Descriptor	10
Table 8 Supported Data Removal Mechanism	11
Table 9 Data Removal Mechanism Format field = 0	11
Table 10 Data Removal Mechanism Format field = 1	12
Table 11 TPER_RESET Command	13
Table 12 ComID Assignments	15
Table 13 Supported Tokens	16
Table 14 reset_types	17
Table 15 Properties Requirements	18
Table 16 Admin SP - SPInfo Table Preconfiguration	20
Table 17 Admin SP - SPTemplates Table Preconfiguration	20
Table 18 Admin SP - Table Table Preconfiguration	21
Table 19 Admin SP - MethodID Table Preconfiguration	22
Table 20 Admin SP - AccessControl Table Preconfiguration	23
Table 21 Admin SP - ACE Table Preconfiguration	29
Table 22 Admin SP - Authority Table Preconfiguration	30
Table 23 Admin SP - C_PIN Table Preconfiguration	30
Table 24 Admin SP – TPerInfo Columns	32
Table 25 Admin SP - TPerInfo Table Preconfiguration	32
Table 26 Admin SP - Template Table Preconfiguration	32
Table 27 Admin SP - SP Table Preconfiguration	33
Table 28 data_removal_mechanism Type Table Addition	33
Table 29 data_removal_mechanism Enumeration Values	33
Table 30 DataRemovalMechanism Table Description	34
Table 31 Admin SP - DataRemovalMechanism Table Preconfiguration	34
Table 32 Locking SP - SPInfo Table Preconfiguration	36
Table 33 Locking SP - SPTemplates Table Preconfiguration	36
Table 34 Locking SP - Table Table Preconfiguration	37
Table 35 Locking SP - MethodID Table Preconfiguration	38
Table 36 Locking SP - AccessControl Table Preconfiguration	39
Table 37 Locking SP - ACE Table Preconfiguration	50
Table 38 Locking SP - Authority Table Preconfiguration	52
Table 39 Locking SP - C_PIN Table Preconfiguration	53
Table 40 Locking SP – LockingInfo Columns	55
Table 41 Locking SP - LockingInfo Table Preconfiguration	55
Table 42 Locking SP - Locking Table Preconfiguration	56
Table 43 Locking SP - MBRControl Table Preconfiguration	56
Table 44 LifeCycle Type Table Modification	64
Table 45 Table Table Additional Columns	65

1 Introduction

1.1 Document Purpose

The Storage Workgroup specifications provide a comprehensive architecture for putting Storage Devices under policy control as determined by the trusted platform host, the capabilities of the Storage Device to conform to the policies of the trusted platform, and the lifecycle state of the Storage Device as a Trusted Peripheral.

1.2 Scope and Intended Audience

This specification defines the Pyrite Security Subsystem Class (SSC). Any SD that claims Pyrite SSC compatibility SHALL conform to this specification.

The intended audience for this specification is both trusted Storage Device manufacturers and developers that want to use these Storage Devices in their systems.

1.3 Key Words

Key words are used to signify SSC requirements.

The Key Words “**SHALL**”, “**SHALL NOT**”, “**SHOULD**,” and “**MAY**” are used in this document. These words are a subset of the RFC 2119 key words used by TCG, and have been chosen since they map to key words used in T10/T13 specifications. These key words are to be interpreted as described in [1].

In addition to the above key words, the following are also used in this document to describe the requirements of particular features, including tables, methods, and usages thereof.

- **Mandatory (M):** When a feature is Mandatory, the feature SHALL be implemented. A Compliance test SHALL validate that the feature is operational.
- **Optional (O):** When a feature is Optional, the feature MAY be implemented. If implemented, a Compliance test SHALL validate that the feature is operational.
- **Excluded (X):** When a feature is Excluded, the feature SHALL NOT be implemented. A Compliance test SHALL validate that the feature is not operational.
- **Not Required (N)** When a feature is Not Required, the feature MAY be implemented. No Compliance test is required.

1.4 Document References

- [1]. IETF RFC 2119, 1997, “Key words for use in RFCs to Indicate Requirement Levels”
- [2]. Trusted Computing Group (TCG), “TCG Storage Architecture Core Specification”, Version 2.01
- [3]. Trusted Computing Group (TCG), “TCG Storage Interface Interactions Specification”, Version 1.08
- [4]. Trusted Computing Group (TCG), “TCG Storage Security Subsystem Class: Opal”, Version 2.01
- [5]. Trusted Computing Group (TCG), “TCG Storage Feature Set: Block SID Authentication”, Version 1.00
- [6]. Trusted Computing Group (TCG), “TCG Storage Opal SSC Feature Set: PSID”, Version 1.00

1.5 Document Precedence

In the event of conflicting information in this specification and other documents, the precedence for requirements is:

1. This specification
2. Storage Interface Interactions Specification [3]

3. TCG Storage Architecture Core Specification [2]

1.6 SSC Terminology

This section provides special definitions that are not defined in [2].

Table 1 Pyrite SSC Terminology

Term	Definition
Manufactured SP	A Manufactured SP is an SP that was created and preconfigured during the SD manufacturing process
N/A	Not Applicable.
Original Factory State (OFS)	The original state of an SP when it was created in manufacturing, including its table data, access control settings, and life cycle state. Each Manufactured SP has its own Original Factory State. Original Factory State applies to Manufactured SPs only.
Vendor Unique (VU)	These values are unique to each SD manufacturer. Typically VU is used in table cells.
NN NN	The LSBs of a Locking object's UID (hexadecimal)
(MBR O)	Values in this specification marked with this identifier are components of the MBR Shadowing feature.

1.7 Legend

The following legend defines SP table cell coloring coding. This color coding is informative only. The table cell content is normative.

Table 2 SP Table Legend

Table Cell Legend	R-W	Value	Access Control	Comment
Arial-Narrow	Read-only	Pyrite SSC specified	Fixed	- Cell content is Read-Only. - Access control is fixed. - Value is specified by the Pyrite SSC
<u>Arial Narrow bold-under</u>	Read-only	VU	Fixed	- Cell content is Read-Only. - Access Control is fixed. - Values are Vendor Unique (VU). A minimum or maximum value may be specified.
Arial-Narrow	Not Defined	(N)	Not Defined	- Cell content is (N). - Access control is not defined. - Any text in table cell is informative only. - A <code>Get</code> MAY omit this column from the method response.
<u>Arial Narrow bold-under</u>	Write	Preconfigured, user personalizable	Preconfigured, user personalizable	- Cell content is writable. - Access control is personalizable <code>Get</code> Access Control is not described by this color coding
Arial-Narrow	Write	Preconfigured, user personalizable	Fixed	- Cell content is writable. - Access control is fixed. <code>Get</code> Access Control is not described by this color coding

2 Pyrite SSC Overview

2.1 Pyrite SSC Use Cases and Threats

Begin Informative Content

The Pyrite SSC is an implementation profile for Storage Devices built to:

- Provide logical locking of the storage device interface
- Enable interoperability between multiple SD vendors

A Pyrite SSC compliant SD:

- Facilitates feature discoverability
- Provides some user definable features (e.g. access control, user passwords, etc.)
- Supports Pyrite SSC unique behaviors (e.g. communication, table management)

A SD that implements Pyrite SSC provides a mechanism to control access to user data. This SSC does not specify capabilities for cryptographic protection of user data at rest.

This specification addresses a limited set of use cases. They are:

- Deploy Storage Device & Take Ownership: the Storage Device is integrated into its target system and ownership transferred by setting or changing the Storage Device's owner credential.
- Activate or Enroll Storage Device: access control credentials (re)generated and/or set on the Storage Device. Access control is configured for LBA range unlocking.
- Lock & Unlock Storage Device: unlocking and locking under host control via either an explicit lock or implicit lock triggered by a reset event.
- Repurpose & End-of-Life Preparation: reset of locking credential(s) and user data removal for Storage Device repurposing or decommissioning.

This specification is defined as a limited subset of [4], with the intent to provide a smaller set of features and use cases while remaining command compatible, but without mandating support of media encryption, such that an application that can manage a Pyrite SSC-based SD can also manage a SD that is compatible with [4].

End Informative Content

2.2 Security Providers (SPs)

A Pyrite SSC compliant SD SHALL support at least two Security Providers (SPs):

- 1) Admin SP
- 2) Locking SP

The Locking SP MAY be created by the SD manufacturer.

2.3 Interface Communication Protocol

A Pyrite SSC compliant SD SHALL implement the synchronous communications protocol as defined in section 3.3.4.

This communication protocol operates based upon configuration information defined by:

- 1) The values reported via Level 0 Discovery (section 3.1.1)
- 2) The combination of the host's communication properties and the TPer's communication properties (see Properties Method section 4.1.1.1)

2.4 Authentication

A Pyrite SSC compliant SD SHALL support password authorities and authentication.

2.5 Table Management

This specification defines the mandatory tables and mandatory/optional table rows delivered by the SD manufacturer. The creation or deletion of tables after manufacturing is outside the scope of this specification. The creation or deletion of table rows post-manufacturing is outside the scope of this specification.

2.6 Access Control & Personalization

Initial access control policies are preconfigured at SD manufacturing time on manufacturer created SPs. A Pyrite SSC compliant SD SHALL support personalization of certain Access Control Elements of the Locking SP.

2.7 Issuance

The Locking SP MAY be present in the SD when the SD leaves the manufacturer. The issuance of SPs is outside the scope of this specification.

2.8 SSC Discovery

Refer to [2] for details (see section 3.1.1).

2.9 MBR Shadowing

“MBR Shadowing” is a feature that allows loading and execution of pre(OS)-boot code that is used to unlock LBA ranges. This feature and its components are Optional in this specification.

MBR Shadowing components are marked with the identifier “(MBR O)”.

If the MBR Shadowing feature is supported, then all components of this feature as identified by “(MBR O)” SHALL be supported, and Level 0 discovery SHALL report that MBR Shadowing is supported.

If the MBR Shadowing feature is not supported, then components identified by “(MBR O)” MAY be absent, and Level 0 discovery SHALL report that MBR Shadowing is not supported.

2.10 Mandatory Feature Sets

A Pyrite SSC compliant SD SHALL support the following TCG Storage Feature Sets:

- Block SID Authentication ([5])
- PSID ([6])

3 Pyrite SSC Features

3.1 Security Protocol 1 Support

3.1.1 Level 0 Discovery (M)

Refer to [2] for more details.

A Pyrite SSC compliant SD SHALL return the following Level 0 response:

- Level 0 Discovery Header
- TPer Feature Descriptor
- Locking Feature Descriptor
- Pyrite SSC Feature Descriptor
- Supported Data Removal Mechanism Feature Descriptor

3.1.1.1 Level 0 Discovery Header

Table 3 Level 0 Discovery Header

Bit Byte	7	6	5	4	3	2	1	0
0	(MSB)	Length of Parameter Data						
1								
2								
3								(LSB)
4	(MSB)	Data structure revision						
5								
6								
7								(LSB)
8	(MSB)	Reserved						
...								
15								(LSB)
16	(MSB)	Vendor Specific						
...								
47								(LSB)

- Length of parameter data = VU
- Data structure revision = 0x00000001 or any version that supports the defined features in this SSC
- Vendor Specific = VU

3.1.1.2 TPer Feature (Feature Code = 0x0001)

Table 4 Level 0 Discovery - TPer Feature Descriptor

Bit Byte	7	6	5	4	3	2	1	0
0	(MSB) Feature Code (0x0001) (LSB)							
1								
2	Version				Reserved			
3	Length							
4	Reserved	ComID Mgmt Supported	Reserved	Streaming Supported	Buffer Mgmt Supported	ACK/NAK Supported	Async Supported	Sync Supported
5 - 15	Reserved							

- Feature Code = 0x0001
- Version = 0x1 or any version that supports the defined features in this SSC
- Length = 0x0C
- ComID Mgmt Supported = VU
- Streaming Supported = 1
- Buffer Mgmt Supported = VU
- ACK/NACK Supported = VU
- Async Supported = VU
- Sync Supported = 1

3.1.1.3 Locking Feature (Feature Code = 0x0002)

** = the present current state of the respective feature

Table 5 Level 0 Discovery - Locking Feature Descriptor

Bit Byte	7	6	5	4	3	2	1	0
0	(MSB) Feature Code (0x0002) (LSB)							
1								
2	Version				Reserved			
3	Length							
4	Reserved	MBR Shadowing Not Supported	MBR Done	MBR Enabled	Media Encryption	Locked	Locking Enabled	Locking Supported
5 - 15	Reserved							

- Feature Code = 0x0002
- Version = 0x2 or any version that supports the defined features in this SSC
- Length = 0x0C
- MBR Shadowing Not Supported (MBR O) = VU
 - If MBR Shadowing feature is not absent (i.e., is supported), this bit SHALL be 0.
 - If MBR Shadowing feature is absent (i.e., is not supported), this bit SHALL be 1.
- MBR Done (MBR O) = See section 3.1.1.3.1

- MBR Enabled (MBR O) = See section 3.1.1.3.2
- Media Encryption = 0
- Locked = **
- Locking Enabled = See section 3.1.1.3.3
- Locking Supported = 1

3.1.1.3.1 MBR Done Definition

The definition of the MBR Done bit is changed from [2] as follows:

If

- a) MBR Shadowing Not Supported bit is set to zero;
- b) MBR Enabled bit is set to one; and
- c) MBRControl table's Done column has a value of "True",

then the MBR Done bit SHALL be set to one.

Else, the MBR Done bit SHALL be set to zero.

3.1.1.3.2 MBR Enabled Definition

The definition of the MBR Enabled bit is changed from [2] as follows:

If

- a) MBR Shadowing Not Supported bit is set to zero;
- b) Locking Enabled bit is set to one; and
- c) MBRControl table's Enabled column has a value of "True",

then the MBR Enabled bit SHALL be set to one.

Else, the MBR Enabled bit SHALL be set to zero.

3.1.1.3.3 Locking Enabled Definition

The definition of the Locking Enabled bit is changed from [2] as follows:

The Locking Enabled bit SHALL be set to one if an SP that incorporates the Locking template is any state other than Nonexistent or Manufactured-Inactive; otherwise the Locking Enabled bit SHALL be set to zero.

3.1.1.4 Pyrite SSC V2.00 Feature (Feature Code = 0x0303)

Table 6 Level 0 Discovery - Pyrite SSC Feature Descriptor

Byte	Bit	7	6	5	4	3	2	1	0
0	(MSB)	Feature Code (0x0303)							
1		(LSB)							
2		Version				Reserved			
3		Length							
4	(MSB)	Base ComID							
5		(LSB)							
6	(MSB)	Number of ComIDs							
7		(LSB)							
8-12		Reserved for future common SSC parameters							
13		Initial C_PIN_SID PIN Indicator							
14		Behavior of C_PIN_SID PIN upon TPer Revert							
15-19		Reserved for future common SSC parameters							

- Feature Code = 0x0303
- Version = 0x1 or any version that supports the defined features in this SSC
- Length = 0x10
- Base ComID = VU
- Number of ComIDs >= 1
- Initial C_PIN_SID PIN Indicator = 0x00
 - 0x00 = The initial C_PIN_SID PIN value is equal to the C_PIN_MSID PIN value
- Behavior of C_PIN_SID PIN upon TPer Revert = 0x00
 - 0x00 = The C_PIN_SID PIN value becomes the value of the C_PIN_MSID PIN column after successful invocation of `Revert` on the Admin SP's object in the `SP` table

3.1.1.5 Supported Data Removal Mechanism Feature (Feature Code = 0x0404)

Table 7 Level 0 Discovery – Supported Data Removal Mechanism Feature Descriptor

Byte	Bit	7	6	5	4	3	2	1	0
0	(MSB)	Feature Code (0x0404)							(LSB)
1									
2		Version				Reserved			
3		Length							
4		Reserved							
5		Reserved							Data Removal Operation Processing
6		Supported Data Removal Mechanism							
7		Reserved	Data Removal Time Format for Bit 5	Data Removal Time Format for Bit 4	Data Removal Time Format for Bit 3	Data Removal Time Format for Bit 2	Data Removal Time Format for Bit 1	Data Removal Time Format for Bit 0	
8-9		Data Removal Time for Supported Data Removal Mechanism Bit 0							
10-11		Data Removal Time for Supported Data Removal Mechanism Bit 1							
...		...							
18-19		Data Removal Time for Supported Data Removal Mechanism Bit 5							
20-35		Reserved for future Supported Data Removal Mechanism parameters							

- Feature Code = 0x0404
- Version = 0x1 or any version that supports the defined features in this SSC
- Length = 0x20
- Data Removal Operation Processing = See section 3.1.1.5.1
- Supported Data Removal Mechanism = VU, See section 3.1.1.5.2
- Data Removal Time Format = See section 3.1.1.5.3
- Data Removal Time for Supported Data Removal Mechanism Bit = See section 3.1.1.5.3

3.1.1.5.1 Data Removal Operation Processing Definition

Data Removal Operation Processing SHALL be set to one if the TPer is performing `Revert` or `RevertSP` operation; otherwise, Data Removal Operation Processing SHALL be set to zero.

3.1.1.5.2 Supported Data Removal Mechanism Definition

Each bit SHALL be set to 1 if the TPer supports the corresponding Data Removal Mechanism; otherwise, it SHALL be set to 0. The TPer SHALL support Overwrite Data Erase or Block Erase. The TPer MAY support multiple Data Removal Mechanisms described in Table 8. After a `Revert` method has completed without an

error, or a `RevertSP` method has completed without an error, the condition of user data SHALL be indicated as specified in Table 8.

The TPer SHALL set the Crypto Erase bit to 0.

Table 8 Supported Data Removal Mechanism

Bit	Name	Condition of user data after Data Removal
0	Overwrite Data Erase	The Overwrite Data Erase mechanism causes the TPer to alter information by writing a vendor specific data pattern to the medium.
1	Block Erase	The Block Erase mechanism causes the TPer to alter information by setting the physical blocks to a vendor specific value.
2	Crypto Erase	The Crypto Erase mechanism eradicates the cryptographic keys used to encrypt the user data.
3	Unmap	The Unmap mechanism causes all mapping between LBAs and internal media to be removed.
4	Reset Write Pointers	The Reset Write Pointers mechanism causes all zones (i.e. write pointer zones) to have the write pointer set to the first LBA in that zone.
5	Vendor Specific Erase	The Vendor Specific Erase mechanism causes all user data to be removed by a vendor specific method.
6-7	Reserved	

3.1.1.5.3 Data Removal Time Format and Data Removal Time Definition

Data Removal Time field gives an estimate of the time required to perform the erasure corresponding to the Data Removal Mechanism defined in Supported Data Removal Mechanism field. Data Removal Time Format bit indicates the format used to express the time as follows:

- a) if the Data Removal Time Format bit is 0, then the estimated time is defined in Table 9; and
- b) if the Data Removal Time Format bit is set to 1, then the estimated time is defined in Table 10.

Data Removal Time Format bit and Data Removal Time Format field are defined in Table 9 and Table 10.

Table 9 Data Removal Mechanism Format field = 0

Value	Time
0	Not reported
1..65534	(Value x 2) seconds
65535	> 131068 seconds

Table 10 Data Removal Mechanism Format field = 1

Value	Time
0	Not reported
1..65534	(Value x 2) minutes
65535	> 131068 minutes

Begin Informative Content

Data Removal Time field gives an estimate of the total time required to perform the erasure. This field is not a dynamic estimate of the remaining time for completion.

End Informative Content

3.2 Security Protocol 2 Support

3.2.1 ComID Management

ComID management support is reported in Level 0 Discovery. Statically allocated ComIDs are also discoverable via the Level 0 Discovery response.

3.2.2 Stack Protocol Reset (M)

A Pyrite SSC compliant SD SHALL support the Stack Protocol Reset command. Refer to [2] for details.

3.2.3 TPER_RESET command (M)

If the TPER_RESET command is enabled, the TPER_RESET command SHALL cause the following before the TPer accepts the next IF-SEND or IF-RECV command:

- all dynamically allocated ComIDs SHALL return to the Inactive state;
- all open sessions SHALL be aborted on all ComIDs;
- all uncommitted transactions SHALL be aborted on all ComIDs;
- the synchronous protocol stack for all ComIDs SHALL be reset to its initial state;
- all TCG command and response buffers SHALL be invalidated for all ComIDs;
- all related method processing including the `Revert` and `RevertSP` methods occurring on all ComIDs SHALL be aborted;
- TPer's knowledge of the host's communications capabilities, on all ComIDs, SHALL be reset to the initial minimum assumptions defined in [2] or the TPer's SSC definition;
- the values of the `ReadLocked` and `WriteLocked` columns SHALL be set to `True` for all Locking SP's Locking objects that contain the Programmatic enumeration value in the `LockedOnReset` column;
- if MBR Shadowing is supported and the `DoneOnReset` column contains the Programmatic enumeration value, then the value of the `Done` column of the Locking SP's `MBRControl` table SHALL be set to `False`;

The TPER_RESET command is delivered by the transport IF-SEND command. If the TPER_RESET command is enabled, the TPer SHALL accept and acknowledge it at the interface level. If the TPER_RESET command is disabled, the TPer SHALL abort it at the interface level with the “Other Invalid Command Parameter” status (see [3]). There is no IF-RECV response to the TPER_RESET command.

The TPER_RESET command is defined in Table 11.

The Transfer Length SHALL be non-zero. All data transferred SHALL be ignored.

Table 11 TPER_RESET Command

FIELD	VALUE
Command	IF-SEND
Protocol ID	0x02
Transfer Length	Non-zero
ComID	0x0004

3.3 Communications

3.3.1 Communication Properties

The TPer SHALL support the minimum communication buffer size as defined in Section 4.1.1.1. For each ComID, the physical buffer size SHALL be reported to the host via the `Properties` method.

The TPer SHALL terminate any IF-SEND command whose transfer length is greater than the reported `MaxComPacketSize` size for the corresponding ComID. Refer to in [3] for details about “Invalid Transfer Length parameter on IF-SEND”.

Data generated in response to methods contained within an IF-SEND command payload subpacket (including the required `ComPacket` / `Packet` / `Subpacket` overhead data) SHALL fit entirely within the response buffer. If the method response and its associated protocol overhead do not fit completely within the response buffer, the TPer

- 1) SHALL terminate processing of the IF-SEND command payload,
- 2) SHALL NOT return any part of the method response if the Sync Protocol is being used, and
- 3) SHALL return an empty response list with a TCG status code of `RESPONSE_OVERFLOW` in that method's response status list.

3.3.2 Supported Security Protocols

The TPer SHALL support:

- IF-RECV commands with a Security Protocol values of 0x00, 0x01, 0x02.
- IF-SEND commands with a Security Protocol values of 0x01, 0x02.

3.3.3 ComIDs

For the purpose of communication using Security Protocol 0x01, the TPer SHALL:

- support at least one statically allocated ComID for Synchronous Protocol communication.
- have the ComID Extension values = 0x0000 for all statically allocated ComIDs.
- keep all statically allocated ComIDs in the Active state.

When the TPer receives an IF-SEND or IF-RECV with an inactive or unsupported ComID, the TPer SHALL either:

- terminate the command as defined in [3] with “Other Invalid Command parameter”, or
- follow the requirements defined in [2] for “Inactive or Unsupported ComID parameter on IF-SEND” or “Inactive or Unsupported ComID parameter on IF-RECV”.

ComIDs SHALL be assigned based on the allocation presented in Table 12

Table 12 ComID Assignments

ComID	Description
0x0000	Reserved
0x0001	Level 0 Device Discovery
0x0002-0x0003	Reserved for TCG
0x0004	TPER_RESET command
0x0005-0x07FF	Reserved for TCG
0x0800-0x0FFF	Vendor Unique
0x1000-0xFFFF	ComID management (Protocol ID=0x01 and 0x02)

3.3.4 Synchronous Protocol

The TPer SHALL support the Synchronous Protocol. Refer to [2] for details.

3.3.4.1 Payload Encoding

3.3.4.1.1 Stream Encoding Modifications

The TPer SHALL support tokens listed in Table 13. If an unsupported token is encountered, the TPer SHALL treat this as a streaming protocol violation and return an error per the definition in section 3.3.4.1.3.

Table 13 Supported Tokens

Acronym	Meaning
	Tiny atom
	Short atom
	Medium atom
	Long atom
SL	Start List
EL	End List
SN	Start Name
EN	End Name
CALL	Call
EOD	End of Data
EOS	End of session
ST	Start transaction
ET	End of transaction
MT	Empty atom

The TPer SHALL support the above token atoms with the B bit set to 0 or 1 and the S bit set to 0.

3.3.4.1.2 TCG Packets

Within a single IF-SEND/IF-RECV command, the TPer SHALL support a ComPacket containing one Packet, which contains one Subpacket. The Host MAY discover TPer support of capabilities beyond this requirement in the parameters returned in response to a `Properties` method.

3.3.4.1.3 Payload Error Response

The TPer SHALL respond according to the following rules if it encounters a streaming protocol violation:

- If the error is on Session Manager or is such that the TPer cannot resolve a valid session ID from the payload (i.e. errors in the ComPacket header or Packet header), then the TPer SHALL discard the payload and immediately transition to the “Awaiting IF-SEND” state.
- If the error occurs after the TPer has resolved the session ID, then the TPer SHALL abort the session and MAY prepare a `CloseSession` method for retrieval by the host.

3.3.5 Storage Device Resets

3.3.5.1 Interface Resets

Interface resets that generate TCG reset events are defined in [3].

Interface initiated TCG reset events SHALL result in:

1. All open sessions SHALL be aborted;
2. All uncommitted transactions SHALL be aborted;
3. All pending session startup activities SHALL be aborted;
4. All TCG command and response buffers SHALL be invalidated;
5. All related method processing including the `Revert` and `RevertSP` methods SHALL be aborted;

6. For each ComID, the state of the synchronous protocol stack SHALL transition to “Awaiting IF-SEND” state;
7. No notification of these events SHALL be sent to the host.

3.3.5.2 TCG Reset Events

Table 14 replaces the definition of TCG reset_types that are defined in [2]:

Table 14 reset_types

Enumeration value	Associated Value
0	Power Cycle
1	Hardware
2	HotPlug
3	Programmatic
4-15	Reserved
16-31	Vendor Unique

3.3.6 Protocol Stack Reset Commands (M)

An IF-SEND containing a Protocol Stack Reset Command SHALL be supported.

Refer to [2] for details.

4 Pyrite SSC-compliant Functions and SPs

4.1 Session Manager

4.1.1 Methods

4.1.1.1 Properties (M)

A Pyrite compliant SD SHALL support the `Properties` method. The requirements for support of the various TPer and Host properties, and the requirements for their values, are shown in Table 15.

Table 15 Properties Requirements

Property Name	TPer Property Requirements and Values Reported	Host Property Requirements and Values Accepted
MaxComPacketSize	(M) 2048 minimum	(M) Initial Assumption: 2048 Minimum allowed: 2048 Maximum allowed: VU
MaxResponseComPacketSize	(M) 2048 minimum	(N) Although this is a legal host property, there is no requirement for the TPer to use it. The TPer MAY ignore this host property and not list it in the HostProperties result of the <code>Properties</code> method response.
MaxPacketSize	(M) 2028 minimum	(M) Initial Assumption: 2028 Minimum allowed: 2028 Maximum allowed: VU
MaxIndTokenSize	(M) 1992 minimum	(M) Initial Assumption: 1992 Minimum allowed: 1992 Maximum allowed: VU
MaxPackets	(M) 1 minimum	(M) Initial Assumption: 1 Minimum allowed: 1 Maximum allowed: VU
MaxSubpackets	(M) 1 minimum	(M) Initial Assumption: 1 Minimum allowed: 1 Maximum allowed: VU
MaxMethods	(M) 1 minimum	(M) Initial Assumption: 1 Minimum allowed: 1 Maximum allowed: VU
MaxSessions	(M) 1 minimum	N/A – not a host property
MaxAuthentications	(M) 2 minimum	N/A – not a host property
MaxTransactionLimit	(M) 1 minimum	N/A – not a host property
DefSessionTimeout	(M) VU	N/A – not a host property

4.1.1.2 StartSession (M)

A Pyrite-compliant SD SHALL support the following parameters for the `StartSession` method:

- `HostSessionID`
- `SPID`
- `Write` = support for "True" is (M), support for "False" is (N)
- `HostChallenge`
- `HostSigningAuthority`

4.1.1.3 SyncSession (M)

A Pyrite-compliant SD SHALL support the following parameters for the `SyncSession` method:

- `HostSessionID`
- `SPSessionID`

4.1.1.4 CloseSession (O)

A Pyrite-Compliant SD MAY support the `CloseSession` method.

4.2 Admin SP

The Admin SP includes the Base Template and the Admin Template.

4.2.1 Base Template Tables

All tables included in the following subsections are mandatory.

4.2.1.1 SPInfo (M)

Table 16 Admin SP - SPInfo Table Preconfiguration

UID	SPID	Name	Size	SizeInUse	SPSessionTimeout	Enabled
00 00 00 02 00 00 00 01	00 00 02 05 00 00 00 01	"Admin"				T

4.2.1.2 SPTemplates (M)

*ST1 = this version number or any version number that complies with this SSC.

Table 17 Admin SP - SPTemplates Table Preconfiguration

UID	TemplateID	Name	Version
00 00 00 03 00 00 00 01	00 00 02 04 00 00 00 01	"Base"	00 00 00 02 *ST1
00 00 00 03 00 00 00 02	00 00 02 04 00 00 00 02	"Admin"	00 00 00 02 *ST1

4.2.1.3 Table (M)

Refer to section 5.3 for a description and requirements of the MandatoryWriteGranularity and RecommendedAccessGranularity columns.

Table 18 Admin SP - Table Table Preconfiguration

UID	Name	CommonName	TemplateID	Kind	Column	NumColumns	Rows	RowsFree	RowBytes	LastID	MinSize	MaxSize	MandatoryWriteGranularity	RecommendedAccessGranularity
00 00 00 01 00 00 00 01	"Table"			Object									0	0
00 00 00 01 00 00 00 02	"SPInfo"			Object									0	0
00 00 00 01 00 00 00 03	"SPTemplates"			Object									0	0
00 00 00 01 00 00 00 06	"MethodID"			Object									0	0
00 00 00 01 00 00 00 07	"AccessControl"			Object									0	0
00 00 00 01 00 00 00 08	"ACE"			Object									0	0
00 00 00 01 00 00 00 09	"Authority"			Object									0	0
00 00 00 01 00 00 00 0B	"C_PIN"			Object									0	0
00 00 00 01 00 00 02 01	"TPerInfo"			Object									0	0
00 00 00 01 00 00 02 04	"Template"			Object									0	0
00 00 00 01 00 00 02 05	"SP"			Object									0	0
00 00 00 01 00 00 11 01	"DataRemovalMechanism"			Object									0	0

Begin Informative Content

[2] states, "The `Table` table in the Admin SP includes a row for each table that the TPer supports, in addition to a row for each table that exists in the Admin SP." However, the Pyrite SSC requires only the tables from the Admin SP to be included in the Admin SP's `Table` table, as indicated in Table 18.

End Informative Content

4.2.1.4 MethodID (M)

*MT1 = refer to section 5.1.2 for details on the requirements for supporting *Revert*.

*MT2 = refer to section 5.1.1 for details on the requirements for supporting *Activate*.

Table 19 Admin SP - MethodID Table Preconfiguration

UID	Name	CommonName	TemplateID
00 00 00 06 00 00 00 08	"Next"		
00 00 00 06 00 00 00 0D	"GetACL"		
00 00 00 06 00 00 00 16	"Get"		
00 00 00 06 00 00 00 17	"Set"		
00 00 00 06 00 00 00 1C	"Authenticate"		
00 00 00 06 00 00 02 02 *MT1	"Revert"		
00 00 00 06 00 00 02 03 *MT2	"Activate"		
00 00 00 06 00 00 06 01	"Random"		

4.2.1.5 AccessControl (M)

The following table contains Optional rows designated with (O).

*AC1 = TT TT TT TT is a shorthand for the LSBs of the Table object UIDs

*AC2 = TT TT TT TT is a shorthand for the LSBs of the SPTemplates object UIDs

*AC3 = TT TT TT TT is a shorthand for the LSBs of the MethodID object UIDs

*AC4 = TT TT TT TT is a shorthand for the LSBs of the ACE object UIDs

*AC5 = TT TT TT TT is a shorthand for the LSBs of the Authority object UIDs

*AC6 = TT TT TT TT is a shorthand for the LSBs of the Template object UIDs

*AC7 = TT TT TT TT is a shorthand for the LSBs of the SP object UIDs

*AC8 = refer to section 5.1.2 for details on the requirements for supporting *Revert*

*AC9 = Support for ACE_SP_SID is mandatory and support for ACE_Admin is optional

*AC10 = refer to section 5.1.1 for details on the requirements for supporting *Activate*

Notes:

- The *InvokingID*, *MethodID* and *GetACLACL* columns are a special case. Although they are marked as Read-Only with fixed access control, the access control for invocation of the *Get* method is (N).
- The *ACL* column is readable only via the *GetACL* method.

Table 20 Admin SP - AccessControl Table Preconfiguration

Table association - Informative text	UID	InvokingID	InvokingID Name - informative text	MethodID	CommonName	ACL	Log	AddACEACL	RemoveACEACL	GetACLACL	DeleteMethodACL	AddACELog	RemoveACELog	GetACLLog	DeleteMethodLog	LogTo
Table																
		00 00 00 01 00 00 00 00	Table	Next		ACE_Anybody				ACE_Anybody						
		00 00 00 01 TT TT TT TT *AC1	TableObj	Get		ACE_Anybody				ACE_Anybody						
SPInfo																
		00 00 00 02 00 00 00 01	SPInfoObj	Get		ACE_Anybody				ACE_Anybody						
SPTemplates																
		00 00 00 03 00 00 00 00	SPTemplates	Next		ACE_Anybody				ACE_Anybody						
		00 00 00 03 TT TT TT TT *AC2	SPTemplatesObj	Get		ACE_Anybody				ACE_Anybody						
MethodID																
		00 00 00 06 00 00 00 00	MethodID	Next		ACE_Anybody				ACE_Anybody						
		00 00 00 06 TT TT TT TT *AC3	MethodIDObj	Get		ACE_Anybody				ACE_Anybody						

Table association - Informative text	UID	InvokingID	InvokingID Name - informative text	MethodID	CommonName	ACL	Log	AddACEACL	RemoveACEACL	GetACLACL	DeleteMethodACL	AddACELog	RemoveACELog	GetACLLog	DeleteMethodLog	LogTo
ACE																
		00 00 00 08 00 00 00 00	ACE	Next		ACE_Anybody				ACE_Anybody						
		00 00 00 08 TT TT TT TT *AC4	ACEObj	Get		ACE_Anybody				ACE_Anybody						
Authority																
		00 00 00 09 00 00 00 00	Authority	Next		ACE_Anybody				ACE_Anybody						
		00 00 00 09 TT TT TT TT *AC5	AuthorityObj	Get		ACE_Anybody				ACE_Anybody						
(0)		00 00 00 09 00 00 02 01	Admin1	Set		ACE_Set_Enabled				ACE_Anybody						
(0)		00 00 00 09 00 00 02 00 (+XX)	AdminXX	Set		ACE_Set_Enabled				ACE_Anybody						
C_PIN																
		00 00 00 0B 00 00 00 00	C_PIN	Next		ACE_Anybody				ACE_Anybody						

Page 25

Table association - Informative text	UID	InvokingID	InvokingID Name - informative text	MethodID	CommonName	ACL	Log	AddACEACL	RemoveACEACL	GetACLACL	DeleteMethodACL	AddACELog	RemoveACELog	GetACLLog	DeleteMethodLog	LogTo
(O)		00 00 00 0B 00 00 02 00 (+XX)	C_PIN_AdminXX	Get		ACE_C_PIN_SID_Get_NOPIN				ACE_Anybody						
(O)		00 00 00 0B 00 00 02 01	C_PIN_Admin1	Set		ACE_C_PIN_Admins_Set_PIN				ACE_Anybody						
(O)		00 00 00 0B 00 00 02 00 (+XX)	C_PIN_AdminXX	Set		ACE_C_PIN_Admins_Set_PIN				ACE_Anybody						
TPerInfo																
		00 00 02 01 00 03 00 01	TPerInfoObj	Get		ACE_Anybody				ACE_Anybody						

Table association - Informative text	UID	InvokingID	InvokingID Name - informative text	MethodID	CommonName	ACL	Log	AddACEACL	RemoveACEACL	GetACLACL	DeleteMethodACL	AddACELog	RemoveACELog	GetACLLog	DeleteMethodLog	LogTo
		00 00 02 01 00 03 00 01	TPerInfoObj	Set		ACE_TPerInfo_Set_ProgrammaticResetEnable				ACE_Anybody						
Template																
		00 00 02 04 00 00 00 00	Template	Next		ACE_Anybody				ACE_Anybody						
		00 00 02 04 TT TT TT TT *AC6	TemplateObj	Get		ACE_Anybody				ACE_Anybody						
SP																
		00 00 00 00 00 00 00 01	ThisSP	Authenticate		ACE_Anybody				ACE_Anybody						
		00 00 00 00 00 00 00 01	ThisSP	Random		ACE_Anybody				ACE_Anybody						
		00 00 02 05 00 00 00 00	SP	Next		ACE_Anybody										

Table association - Informative text	UID	InvokingID	InvokingID Name - informative text	MethodID	CommonName	ACL	Log	AddACEACL	RemoveACEACL	GetACLACL	DeleteMethodACL	AddACELog	RemoveACELog	GetACLLog	DeleteMethodLog	LogTo
		00 00 02 05 TT TT TT TT *AC7	SPObj	Get		ACE_Anybody				ACE_Anybody						
*AC8		00 00 02 05 TT TT TT TT *AC7	SPObj	Revert		ACE_SP_SID, ACE_Admin *AC9				ACE_Anybody						
*AC10		00 00 02 05 TT TT TT TT *AC7	SPObj	Activate		ACE_SP_SID				ACE_Anybody						
DataRemovalMechanism																
		00 00 11 01 00 00 00 01	DataRemovalMechanismObj	Get		ACE_Anybody				ACE_Anybody						
		00 00 11 01 00 00 00 01	DataRemovalMechanismObj	Set		ACE_DataRemovalMechanism_Set _ActiveDataRemovalMechanism				ACE_Anybody						

4.2.1.6 ACE (M)

The following table contains Optional rows designated with (O).

*ACE1 = This row is (M) if the TPer supports either *Activate* or *Revert*, and (N) otherwise.

Table 21 Admin SP - ACE Table Preconfiguration

Table Association - Informative text	UID	Name	CommonName	BooleanExpr	Columns
BaseACEs					
	00 00 00 08 00 00 00 01	"ACE_Anybody"		Anybody	All
(O)	00 00 00 08 00 00 00 02	"ACE_Admin"		Admins	All
Authority					
(O)	00 00 00 08 00 03 00 01	"ACE_Set_Enabled"		SID	Enabled
C_PIN					
	00 00 00 08 00 00 8C 02	"ACE_C_PIN_SID_Get_NOPIN"		Admins OR SID	UID, CharSet, TryLimit, Tries, Persistence
	00 00 00 08 00 00 8C 03	"ACE_C_PIN_SID_Set_PIN"		SID	PIN
	00 00 00 08 00 00 8C 04	"ACE_C_PIN_MSID_Get_PIN"		Anybody	UID, PIN
(O)	00 00 00 08 00 03 A0 01	"ACE_C_PIN_AdminSet_Set_PIN"		Admins OR SID	PIN
TPerInfo					
	00 00 00 08 00 03 00 03	"ACE_TPerInfo_Set_ProgrammaticResetEnable"		SID	ProgrammaticResetEnable
SP					
*ACE1	00 00 00 08 00 03 00 02	"ACE_SP_SID"		SID	All
DataRemoval Mechanism					
*ACE1	00 00 00 08 00 05 00 01	"ACE_DataRemovalMechanism_Set_ActiveDataRemovalMechanism"		Admins OR SID	ActiveDataRemoval Mechanism

4.2.1.7 Authority (M)

The following table contains Optional rows designated with (O).

Table 22 Admin SP - Authority Table Preconfiguration

UID	Name	CommonName	IsClass	Class	Enabled	Secure	HashAndSign	PresentCertificate	Operation	Credential	ResponseSign	ResponseExch	ClockStart	ClockEnd	Limit	Uses	Log	LogTo
00 00 00 09 00 00 00 01	"Anybody"		F	Null	T	None	None	F	None	Null	Null	Null						
00 00 00 09 00 00 00 02	"Admins"		T	Null	T	None	None	F	None	Null	Null	Null						
00 00 00 09 00 00 00 06	"SID"		F	Null	T	None	None	F	Password	C_PIN_SID	Null	Null						
00 00 00 09 00 00 02 01 (O)	"Admin1"		F	Admins	F	None	None	F	Password	C_PIN_Admin1	Null	Null						
00 00 00 09 00 00 02 00 (+XX) (O)	"AdminXX"		F	Admins	F	None	None	F	Password	C_PIN_AdminXX	Null	Null						

4.2.1.8 C_PIN (M)

The following table contains Optional rows designated with (O).

Table 23 Admin SP - C_PIN Table Preconfiguration

UID	Name	CommonName	PIN	CharSet	TryLimit	Tries	Persistence
00 00 00 0B 00 00 00 01	"C_PIN_SID"		MSID	Null	<u>VU</u>	<u>VU</u>	FALSE
00 00 00 0B 00 00 84 02	"C_PIN_MSID"		<u>MSID</u>				
00 00 00 0B 00 00 02 01 (O)	"C_PIN_Admin1"		" "	Null	<u>0</u>	<u>0</u>	FALSE
00 00 00 0B 00 00 02 00 (+XX) (O)	"C_PIN_AdminXX"		" "	Null	<u>0</u>	<u>0</u>	FALSE

The PIN column value of C_PIN_SID is set to the PIN column value of C_PIN_MSID in OFS.

4.2.2 Base Template Methods

Refer to section 4.2.1.4 for supported methods.

4.2.3 Admin Template Tables

4.2.3.1 TPerInfo (M)

The TPerInfo table has the following columns, in addition to those defined in [2]:

Table 24 Admin SP – TPerInfo Columns

Column Number	Column Name	IsUnique	Column Type
0x08	ProgrammaticResetEnable		boolean

- ProgrammaticResetEnable**
 This column indicates whether support for programmatic resets is enabled or not. If ProgrammaticResetEnable is TRUE, then the TPER_RESET command is enabled. If ProgrammaticResetEnable is FALSE, then the TPER_RESET command is not enabled. This column is readable by Anybody and modifiable by the SID authority.

*TP1 = the value in the GUID column SHALL comply with the format defined in [2].

*TP2 = this version or any version that supports the defined features in this SSC.

*TP3 = the SSC column is a list of names and SHALL have "Pyrite" as one of the list elements.

Table 25 Admin SP - TPerInfo Table Preconfiguration

UID	Bytes	GUID	Generation	Firmware Version	ProtocolVersion	SpaceForIssuance	SSC	ProgrammaticResetEnable
00 00 02 01 00 03 00 01		VU *TP1			1 *TP2		["Pyrite"] *TP3	FALSE

4.2.3.2 Template (M)

Table 26 Admin SP - Template Table Preconfiguration

UID	Name	Revision Number	Instances	MaxInstances
00 00 02 04 00 00 00 01	"Base"	1	<u>VU</u>	<u>VU</u>
00 00 02 04 00 00 00 02	"Admin"	1	1	1
00 00 02 04 00 00 00 06	"Locking"	1	1	1

4.2.3.3 SP (M)

*SP1 = this row only exists in the Admin SP's OFS when the Locking SP is created by the manufacturer.

Table 27 Admin SP - SP Table Preconfiguration

UID	Name	ORG	EffectiveAuth	DateOfIssue	Bytes	LifeCycle	Frozen
00 00 02 05 00 00 00 01	"Admin"					Manufactured	FALSE
00 00 02 05 00 00 00 02 *SP1	"Locking"					Manufactured-Inactive	FALSE

4.2.4 Admin Template Methods

Refer to section 4.2.1.4 for supported methods.

4.2.5 Pyrite Additional Column Types

4.2.5.1 data_removal_mechanism

The `data_removal_mechanism` type is defined as follows for Pyrite:

Table 28 data_removal_mechanism Type Table Addition

UID	Name	Format
00 00 00 05 00 00 04 20	data_removal_mechanism	Enumeration_Type, 0, 7

Table 29 defines the enumeration values. The mechanisms associated with each Enumeration Value are defined in Table 8.

Table 29 data_removal_mechanism Enumeration Values

Enumeration Value	Associated Value
0	Overwrite Data Erase
1	Block Erase
2	Crypto Erase
3	Unmap
4	Reset Write Pointers
5	Vendor Specific Erase
6-7	Reserved

4.2.6 Pyrite Additional Data Structures

4.2.6.1 DataRemovalMechanism (Object Table)

The DataRemovalMechanism table is defined in Table 30.

Table 30 DataRemovalMechanism Table Description

Column Number	Column Name	IsUnique	Column Type
0x00	UID		uid
0x01	ActiveDataRemovalMechanism		data_removal_mechanism

4.2.6.1.1 UID

This is the unique identifier of this row in the DataRemovalMechanism table.

This column SHALL NOT be modifiable by the host.

4.2.6.1.2 ActiveDataRemovalMechanism

This column value selects which Data Removal Mechanism in the Supported Data Removal Mechanism field in the Supported Data Removal Mechanism Feature Descriptor is active and will be used to remove data upon execution of the `Revert` method or `RevertSP` method. If an attempt is made to set the ActiveDataRemovalMechanism column value to an unsupported value of the `data_removal_mechanism` enum type, then the Set method invocation SHALL result in the method failing with the status `INVALID_PARAMETER`.

4.2.7 Pyrite Additional Tables

4.2.7.1 DataRemovalMechanism (M)

The DataRemovalMechanism table SHALL contain exactly one row with `UID=0x00 0x00 0x11 0x01 0x00 0x00 0x00 0x01`. The DataRemovalMechanism table SHALL be supported.

Table 31 Admin SP - DataRemovalMechanism Table Preconfiguration

UID	ActiveData Removal Mechanism
00 00 11 01 00 00 00 01	<u>VU</u>

4.2.8 Crypto Template Tables

A Pyrite SSC compliant SD is not required to support any Crypto template tables.

4.2.9 Crypto Template Methods

Refer to section 4.2.1.4 for supported methods.

4.2.9.1 Random

The TPer SHALL implement the `Random` method with the constraints stated in this subsection. TPer support of the following parameter is mandatory:

- `Count`

Attempts to use unsupported parameters SHALL result in a method failure response with TCG status `INVALID_PARAMETER`. The TPer SHALL support `Count` parameter values less than or equal to 32.

4.3 Locking SP

4.3.1 Base Template Tables

All tables defined with (M) in section titles are mandatory.

4.3.1.1 SPInfo (M)

Table 32 Locking SP - SPInfo Table Preconfiguration

UID	SPID	Name	Size	SizeInUse	SPSessionTimeout	Enabled
00 00 00 02 00 00 00 01	00 00 02 05 00 00 00 02	"Locking"				T

4.3.1.2 SPTemplates (M)

*SP1 = this version number or any number that supports the defined features in this SSC

Table 33 Locking SP - SPTemplates Table Preconfiguration

UID	TemplateID	Name	Version
00 00 00 03 00 00 00 01	00 00 02 04 00 00 00 01	"Base"	00 00 00 02 *SP1
00 00 00 03 00 00 00 02	00 00 02 04 00 00 00 06	"Locking"	00 00 00 02 *SP1

4.3.1.3 Table (M)

Refer to section 5.3 for a description and requirements of the MandatoryWriteGranularity and RecommendedAccessGranularity columns.

Refer to section 2.9 for details on the requirements for supporting tables identified with "(MBR O)".

Table 34 Locking SP - Table Table Preconfiguration

UID	Name	CommonName	TemplateID	Kind	Column	NumColumns	Rows	RowsFree	RowBytes	LastID	MinSize	MaxSize	MandatoryWrite Granularity	RecommendedAccess Granularity
00 00 00 01 00 00 00 01	"Table"			Object									0	0
00 00 00 01 00 00 00 02	"SPInfo"			Object									0	0
00 00 00 01 00 00 00 03	"SPTemplates"			Object									0	0
00 00 00 01 00 00 00 06	"MethodID"			Object									0	0
00 00 00 01 00 00 00 07	"AccessControl"			Object									0	0
00 00 00 01 00 00 00 08	"ACE"			Object									0	0
00 00 00 01 00 00 00 09	"Authority"			Object									0	0
00 00 00 01 00 00 00 0B	"C_PIN"			Object									0	0
00 00 00 01 00 00 08 01	"LockingInfo"			Object									0	0
00 00 00 01 00 00 08 02	"Locking"			Object									0	0
00 00 00 01 00 00 08 03 (MBR O)	"MBRControl"			Object									0	0
00 00 00 01 00 00 08 04 (MBR O)	"MBR"			Byte			<u>0x08000000</u> <u>min</u>						<u>VU</u>	<u>VU</u>
00 00 00 01 00 00 10 01	"DataStore"			Byte			<u>0x00020000</u> <u>min</u>						<u>VU</u>	<u>VU</u>

4.3.1.4 Type (N)

The `Type` table is (N) by Pyrite. The following types as defined by [2] SHALL meet the following requirements:

- The "boolean_ACE" type (00000005 000040E) SHALL include the OR Boolean operator.
- The "AC_element" type (00000005 00000801) SHALL support at least 5 entries (2 User authorities, 1 Admin authority, and 2 Boolean operators).
- If a bit in Supported Data Removal Mechanism field of the Supported Data Removal Mechanism Feature Descriptor is set to 1, then the corresponding data_removal_mechanism enumeration value SHALL be supported.

4.3.1.5 MethodID (M)

*MT1 = refer to section 5.1.3 for details on the requirements for supporting `RevertSP`.

Table 35 Locking SP - MethodID Table Preconfiguration

UID	Name	CommonName	TemplateID
00 00 00 06 00 00 00 08	"Next"		
00 00 00 06 00 00 00 0D	"GetACL"		
00 00 00 06 00 00 00 11 *MT1	"RevertSP"		
00 00 00 06 00 00 00 16	"Get"		
00 00 00 06 00 00 00 17	"Set"		
00 00 00 06 00 00 00 1C	"Authenticate"		
00 00 00 06 00 00 06 01	"Random"		

4.3.1.6 AccessControl (M)

The following table contains Optional rows designated with (O).

*AC1 = refer to section 5.1.3 for details on the requirements for supporting `RevertSP`

*AC2 = TT TT TT TT is a shorthand for the LSBs of the Table object UIDs

*AC3 = TT TT TT TT is a shorthand for the LSBs of the SPTemplates object UIDs

*AC4 = TT TT TT TT is a shorthand for the LSBs of the MethodID object UIDs

*AC5 = TT TT TT TT is a shorthand for the LSBs of the ACE object UIDs

*AC6 = TT TT TT TT is a shorthand for the LSB of the Authority object UIDs

(MBR O) = Refer to section 2.9 for requirements for supporting rows marked by this identifier.

Notes:

- The `InvokingID`, `MethodID` and `GetACLACL` columns are a special case. Although they are marked as Read-Only with fixed access control, the access control for invocation of the `Get` method is (N).
- The `ACL` column is readable only via the `GetACL` method.

Table 36 Locking SP - AccessControl Table Preconfiguration

Table Association - informative only	UID	InvokingID	InvokingID Name - informative only	MethodID	CommonName	ACL	Log	AddACEACL	RemoveACEACL	GetACLACL	DeleteMethodACL	AddACELog	RemoveACELog	GetACLLog	DeleteMethodLog	LogTo
SP																
		00 00 00 00 00 00 00 01	ThisSP	Authenticate		ACE_Anybody				ACE_Anybody						
		00 00 00 00 00 00 00 01	ThisSP	Random		ACE_Anybody				ACE_Anybody						
*AC1		00 00 00 00 00 00 00 01	ThisSP	RevertSP		ACE_Admin				ACE_Anybody						
Table																
		00 00 00 01 00 00 00 00	Table	Next		ACE_Anybody				ACE_Anybody						
		00 00 00 01 TT TT TT TT *AC2	TableObj	Get		ACE_Anybody				ACE_Anybody						
SPInfo																
		00 00 00 02 00 00 00 01	SPInfoObj	Get		ACE_Anybody				ACE_Anybody						
SPTemplates																
		00 00 00 03 00 00 00 00	SPTemplates	Next		ACE_Anybody				ACE_Anybody						

Page 40

Table Association - informative only				
UID				
InvokingID	00 00 00 08 00 03 90 00	00 00 00 08 00 03 A8 01	00 00 00 08 00 03 A8 02	00 00 00 08 00 03 D0 00
InvokingID Name - informative only	ACE_Authority_Get_All	ACE_C_PIN_User1_Set_PIN	ACE_C_PIN_User2_Set_PIN	ACE_Locking_GlobalRange_Get_ RangeStartToActiveKey
MethodID	Set	Set	Set	Set
CommonName				
ACL	ACE_ACE_Set_BooleanExpression	ACE_ACE_Set_BooleanExpression	ACE_ACE_Set_BooleanExpression	ACE_ACE_Set_BooleanExpression
Log				
AddACEACL				
RemoveACEACL				
GetACLACL	ACE_Anybody	ACE_Anybody	ACE_Anybody	ACE_Anybody
DeleteMethodACL				
AddACELog				
RemoveACELog				
GetACLLog				
DeleteMethodLog				
LogTo				

(MBR 0)			Table Association - informative only
00 00 00 08 00 03 F8 01			UID
00 00 00 08 00 03 F8 01	00 00 00 08 00 03 E8 00	00 00 00 08 00 03 E0 00	InvokingID
ACE_MBRControl_Set_DoneToDOR	ACE_Locking_GlobalRange_Set_WrLocked	ACE_Locking_GlobalRange_Set_RdLocked	InvokingID Name - informative only
Set	Set	Set	MethodID
			CommonName
ACE_ACE_Set_BooleanExpression	ACE_ACE_Set_BooleanExpression	ACE_ACE_Set_BooleanExpression	ACL
			Log
			AddACEACL
			RemoveACEACL
ACE_Anybody	ACE_Anybody	ACE_Anybody	GetACLACL
			DeleteMethodACL
			AddACELog
			RemoveACELog
			GetACLLLog
			DeleteMethodLog
			LogTo

				Table Association - informative only
				UID
00 00 00 09 TT TT TT TT *AC6	00 00 00 09 00 00 00 00	00 00 00 08 00 03 FC 01	00 00 00 08 00 03 FC 00	InvokingID
AuthorityObj	Authority	ACE_DataStore_Set_All	ACE_DataStore_Get_All	InvokingID Name - informative only
Get	Next	Set	Set	MethodID
				CommonName
ACE_Authority_Get_All	ACE_Anybody	ACE_ACE_Set_BooleanExpression	ACE_ACE_Set_BooleanExpression	ACL
				Log
				AddACEACL
				RemoveACEACL
ACE_Anybody	ACE_Anybody	ACE_Anybody	ACE_Anybody	GetACLACL
				DeleteMethodACL
				AddACELog
				RemoveACELog
				GetACLLog
				DeleteMethodLog
				LogTo

Page 44

Table Association - informative only			
UID			
InvokingID	00 00 00 0B 00 01 00 01	00 00 00 0B 00 01 00 00 (+XX XX)	00 00 00 0B 00 01 00 01
InvokingID Name - informative only	C_PIN_User1	C_PIN_AdminXXXX	C_PIN_Admin1
MethodID	Get	Get	Get
CommonName			
ACL	ACE_C_PIN_Admins_Get_All_NOPIN	ACE_C_PIN_Admins_Get_All_NOPIN	ACE_C_PIN_Admins_Get_All_NOPIN
Log			
AddACEACL			
RemoveACEACL			
GetACLACL	ACE_Anybody	ACE_Anybody	ACE_Anybody
DeleteMethodACL			
AddACELog			
RemoveACELog			
GetACLLLog			
DeleteMethodLog			
LogTo			

Table Association - informative only				
UID				
InvokingID	00 00 00 0B 00 03 00 01	00 00 00 0B 00 01 00 00 (+XX XX)	00 00 00 0B 00 01 00 01	00 00 00 0B 00 03 00 02
InvokingID Name - informative only	C_PIN_User1	C_PIN_AdminXXXX	C_PIN_Admin1	C_PIN_User2
MethodID	Set	Set	Set	Get
CommonName				
ACL	ACE_C_PIN_User1_Set_PIN	ACE_C_PIN_Admins_Set_PIN	ACE_C_PIN_Admins_Set_PIN	ACE_C_PIN_Admins_Get_All_NOPIN
Log				
AddACEACL				
RemoveACEACL				
GetACLACL	ACE_Anybody	ACE_Anybody	ACE_Anybody	ACE_Anybody
DeleteMethodACL				
AddACELog				
RemoveACELog				
GetACLLog				
DeleteMethodLog				
LogTo				

Page 47

Table Association - informative only	UID	InvokingID	InvokingID Name - informative only	MethodID	CommonName	ACL	Log	AddACEACL	RemoveACEACL	GetACLACL	DeleteMethodACL	AddACELog	RemoveACELog	GetACLLog	DeleteMethodLog	LogTo
		00 00 08 02 00 00 00 01	Locking_GlobalRange	Set		ACE_Locking_GlbRng_Admins_Set, ACE_Locking_GlobalRange_Set_RdLocked, ACE_Locking_GlobalRange_Set_WrLocked				ACE_Anybody						
MBRControl																
(MBR 0)		00 00 08 03 00 00 00 01	MBRControlObj	Get		ACE_Anybody				ACE_Anybody						
(MBR 0)		00 00 08 03 00 00 00 01	MBRControlObj	Set		ACE_MBRControl_Admins_Set, ACE_MBRControl_Set_DoneToDOR				ACE_Anybody						
MBR																
(MBR 0)		00 00 08 04 00 00 00 00	MBR	Get		ACE_Anybody				ACE_Anybody						

Page 49

4.3.1.7 ACE (M)

*ACE1 = The TPer SHALL support the values of “Admins” and “Admins OR User1” in the BooleanExpr column of the ACE_C_PIN_User1_Set_PIN ACE. The TPer SHALL fail the Set method invocation with status INVALID_PARAMETER if the host attempts to set a value not supported by the TPer.

*ACE2 = The TPer SHALL support the values of “Admins” and “Admins OR User2” in the BooleanExpr column of the ACE_C_PIN_User2_Set_PIN ACE. The TPer SHALL fail the Set method invocation with status INVALID_PARAMETER if the host attempts to set a value not supported by the TPer.

(MBR O) = Refer to section 2.9 for requirements for supporting rows marked by this identifier.

Table 37 Locking SP - ACE Table Preconfiguration

Table Association -Informative Column	UID	Name	CommonName	BooleanExpr	Columns
Base ACEs					
	00 00 00 08 00 00 00 01	"ACE_Anybody"		Anybody	All
	00 00 00 08 00 00 00 02	"ACE_Admin"		Admins	All
ACE					
	00 00 00 08 00 03 80 00	"ACE_ACE_Get_All"		Admins	All
	00 00 00 08 00 03 80 01	"ACE_ACE_Set_BooleanExpression"		Admins	BooleanExpr
Authority					
	00 00 00 08 00 03 90 00	"ACE_Authority_Get_All"		Admins	All
	00 00 00 08 00 03 90 01	"ACE_Authority_Set_Enabled"		Admins	Enabled
C_PIN					
	00 00 00 08 00 03 A0 00	"ACE_C_PIN_Admins_Get_All_NOPIN"		Admins	UID, CharSet, TryLimit, Tries, Persistence
	00 00 00 08 00 03 A0 01	"ACE_C_PIN_Admins_Set_PIN"		Admins	PIN
	00 00 00 08 00 03 A8 01	"ACE_C_PIN_User1_Set_PIN"		Admins OR User1 *ACE1	PIN
	00 00 00 08 00 03 A8 02	"ACE_C_PIN_User2_Set_PIN"		Admins OR User2 *ACE2	PIN
Locking					
	00 00 00 08 00 03 D0 00	"ACE_Locking_GlobalRange_Get_RangeStartToActiveKey"		Admins	RangeStart, RangeLength, ReadLockEnabled, WriteLockEnabled, ReadLocked, WriteLocked, LockOnReset, ActiveKey

Table Association -Informative Column	UID	Name	CommonName	BooleanExpr	Columns
	00 00 00 08 00 03 E0 00	"ACE_Locking_GlobalRange_Set_RdLocked"		Admins	ReadLocked
	00 00 00 08 00 03 E8 00	"ACE_Locking_GlobalRange_Set_WrLocked"		Admins	WriteLocked
	00 00 00 08 00 03 F0 00	"ACE_Locking_GlbIRng_Admins_Set"		Admins	ReadLockEnabled, WriteLockEnabled, ReadLocked, WriteLocked, LockOnReset
MBRControl					
(MBR O)	00 00 00 08 00 03 F8 00	"ACE_MBRControl_Admins_Set"		Admins	Enable, Done, DoneOnReset
(MBR O)	00 00 00 08 00 03 F8 01	"ACE_MBRControl_Set_DoneToDOR"		Admins	Done, DoneOnReset
DataStore					
	00 00 00 08 00 03 FC 00	"ACE_DataStore_Get_All"		Admins	All
	00 00 00 08 00 03 FC 01	"ACE_DataStore_Set_All"		Admins	All

4.3.1.8 Authority (M)

The following table contains Optional rows designated with (O).

Notes:

- Admin1 is required. Any additional Admin authorities are (O).
- User1 and User2 are required.

Table 38 Locking SP - Authority Table Preconfiguration

UID	Name	CommonName	IsClass	Class	Enabled	Secure	HashAndSign	PresentCertificate	Operation	Credential	ResponseSign	ResponseExch	ClockStart	ClockEnd	Limit	Uses	Log	LogTo
00 00 00 09 00 00 00 01	"Anybody"	"	F	Null	T	None	None	F	None	Null	Null	Null						
00 00 00 09 00 00 00 02	"Admins"	"	T	Null	T	None	None	F	None	Null	Null	Null						
00 00 00 09 00 01 00 01	"Admin1"	"	F	Admins	T	None	None	F	Password	C_PIN_Admin1	Null	Null						
00 00 00 09 00 01 00 00 (+XX XX) (O)	"AdminXXXX"	"	F	Admins	F	None	None	F	Password	C_PIN_AdminXXXX	Null	Null						
00 00 00 09 00 03 00 00	"Users"	"	T	Null	T	None	None	F	None	Null	Null	Null						
00 00 00 09 00 03 00 01	"User1"	"	F	Users	F	None	None	F	Password	C_PIN_User1	Null	Null						
00 00 00 09 00 03 00 02	"User2"	"	F	Users	F	None	None	F	Password	C_PIN_User2	Null	Null						

4.3.1.9 C_PIN (M)

The following table contains Optional rows designated with (O).

Table 39 Locking SP - C_PIN Table Preconfiguration

UID	Name	CommonName	PIN	CharSet	TryLimit	Tries	Persistence
00 00 00 0B 00 01 00 01	"C_PIN_Admin1"		SID	Null	<u>0</u>	<u>0</u>	FALSE
00 00 00 0B 00 01 00 00 (+XX XX) (O)	"C_PIN_AdminXXXX"		""	Null	<u>0</u>	<u>0</u>	FALSE
00 00 00 0B 00 03 00 01	"C_PIN_User1"		""	Null	<u>0</u>	<u>0</u>	FALSE
00 00 00 0B 00 03 00 02	"C_PIN_User2"		""	Null	<u>0</u>	<u>0</u>	FALSE

4.3.2 Base Template Methods

Refer to section 4.3.1.5 for supported methods.

4.3.3 Crypto Template Tables

A Pyrite SSC compliant SD is not required to support any Crypto template tables.

4.3.4 Crypto Template Methods

Refer to section 4.3.1.5 for supported methods.

4.3.4.1 Random

Refer to section 4.2.9.1 for additional constraints imposed on the `Random` method.

4.3.5 Locking Template Tables

4.3.5.1 LockingInfo (M)

The LockingInfo table has the following columns, in addition to those defined in [2]:

Table 40 Locking SP – LockingInfo Columns

Column Number	Column Name	IsUnique	Column Type
0x07	AlignmentRequired		boolean
0x08	LogicalBlockSize		uinteger 4
0x09	AlignmentGranularity		uinteger 8
0x0A	LowestAlignedLBA		uinteger 8

- AlignmentRequired**
 This column indicates whether the TPer requires ranges in the Locking table to be aligned. If AlignmentRequired is TRUE, then the TPer requires ranges to be aligned. If AlignmentRequired is FALSE, then the TPer does not require ranges to be aligned.
 This column SHALL NOT be modifiable by the host and may be retrieved by Anybody.
- LogicalBlockSize**
 This column indicates the number of bytes in a logical block.
 This column SHALL NOT be modifiable by the host and may be retrieved by Anybody.
- AlignmentGranularity**
 This column indicates the number of logical blocks in a group, for alignment purposes (see [4] for a detailed explanation).
 This column SHALL NOT be modifiable by the host and may be retrieved by Anybody.
- LowestAlignedLBA**
 This column indicates the lowest logical block address that is located at the beginning of an alignment granularity group (see [4] for a detailed explanation).
 This column SHALL NOT be modifiable by the host and may be retrieved by Anybody.

Table 41 Locking SP - LockingInfo Table Preconfiguration

UID	Name	Version	EncryptSupport	MaxRanges	MaxReEncryptions	KeysAvailableCfg	AlignmentRequired	LogicalBlockSize	AlignmentGranularity	LowestAlignedLBA
00 00 08 01 00 00 00 01			None	0						

4.3.5.2 Locking (M)

*LT1 = Only a limited set of LockOnReset values is required to be supported by Pyrite SSC SDs. Refer to section 4.3.5.2.1 for details.

Table 42 Locking SP - Locking Table Preconfiguration

UID	Name	CommonName	RangeStart	RangeLength	ReadLockEnabled	WriteLockEnabled	ReadLocked	WriteLocked	LockOnReset	ActiveKey	NextKey	ReEncryptState	ReEncryptRequest	AdvKeyMode	VerifyMode	ContOnReset	LastReEncryptLBA	LastReEncState	GeneralStatus
00 00 08 02 00 00 00 01	"Locking_GlobalRange"	=	0	0	F	F	F	F	Power Cycle *LT1	00 00 00 00 00 00 00 00									

4.3.5.2.1 LockOnReset Restrictions

The TPer SHALL support the following LockOnReset column values:

- a) { 0 } (i.e. Power Cycle); and
- b) { 0, 3 } (i.e. Power Cycle and Programmatic).

4.3.5.3 MBRControl (MBR O)

*MC1 = If MBR Shadowing is supported (i.e., not absent – see section 2.9 and section 3.1.1.3), only a limited set of DoneOnReset values is required to be supported by Pyrite SSC SDs. Refer to section 4.3.5.3.1 for details.

Table 43 Locking SP - MBRControl Table Preconfiguration

UID	Enable	Done	DoneOnReset
00 00 08 03 00 00 00 01	False	<u>False</u>	<u>Power Cycle</u> *MC1

4.3.5.3.1 DoneOnReset Restrictions

The TPer SHALL support the following DoneOnReset column values:

- a) { 0 } (i.e. Power Cycle); and
- b) { 0, 3 } (i.e. Power Cycle and Programmatic).

4.3.5.4 MBR (MBR O)

If MBR Shadowing is supported (i.e., not absent – see section 2.9 and section 3.1.1.3), the MBR minimum size SHALL be 128 MB (0x08000000).

The initial contents of the MBR table SHALL be vendor unique.

4.3.6 Locking Template Methods

Refer to section 4.3.1.5 for supported methods.

4.3.7 Non Template Tables

4.3.7.1 DataStore (M)

The DataStore is a byte table. It can be used by the host for generic secure data storage. The access control for modification or retrieval of data in the table initially requires a member of the Admins class authority. These access control settings are personalizable. Initial DataStore content value is VU.

5 Appendix – SSC Specific Features

5.1 Pyrite SSC-Specific Methods

5.1.1 Activate – Admin Template SP Object Method

`Activate` is a Pyrite SSC-specific method for managing the life cycle of SPs created in manufacturing (Manufactured SP), whose initial life cycle state is “Manufactured-Inactive”.

```
SPObjectUID.Activate[ ]  
=>  
[ ]
```

`Activate` is an object method that operates on objects in the Admin SP's `SP` table. The TPer SHALL NOT permit `Activate` to be invoked on the SP objects of issued SPs.

Invocation of `Activate` on an SP object that is in the “Manufactured-Inactive” state causes the SP to transition to the “Manufactured” state. Invocation of `Activate` on an SP in any other life cycle state SHALL complete successfully provided access control is satisfied, and have no effect. The `Activate` method allows the TPer owner to “turn on” an SP that was created in manufacturing.

This method operates within a Read-Write session to the Admin SP. The SP SHALL be activated immediately after the method returns success if its invocation is not contained within a transaction.

In case of an “Activate Error” (see [3]) `Activate` SHALL fail with a status of FAIL. The MethodID for `Activate` SHALL be 00 00 00 06 00 00 02 03.

5.1.1.1 Activate Support

Support for `Activate` within transactions is (N), and the behavior of `Activate` within transactions is out of the scope of this document.

If the Locking SP was created in manufacturing, and its Original Factory State is Manufactured-Inactive (see section 5.2.2), support for `Activate` on the Locking SP's object in the `SP` Table is mandatory.

5.1.1.2 Side effects of Activate

Upon successful activation of an SP that was in the “Manufactured-Inactive” state, the following changes SHALL be made:

- The `LifeCycleState` column of SP's object in the Admin SP's `SP` table SHALL change to “Manufactured”.
- The current SID PIN (`C_PIN_SID`) in the Admin SP is copied into the `PIN` column of Admin1's `C_PIN` credential (`C_PIN_Admin1`) in the activated SP. This allows for taking ownership of the SP with a known PIN credential.
- Any TPer functionality affected by the life cycle state of the SP based on the templates incorporated into it is modified as defined in the appropriate Template reference section of [2], and as defined in the “State transitions for Manufactured SPs” section (section 5.2.2.2) and “State behaviors for Manufactured SPs” section (section 5.2.2.3) of this specification.

5.1.2 Revert – Admin Template SP Object Method

`Revert` is a Pyrite SSC-specific method for managing the life cycle of SPs created in manufacturing (Manufactured SP).

```
SPObjectUID.Revert[ ]  
=>  
[ ]
```

`Revert` is an object method that operates on objects in the Admin SP's `SP` table. The TPer SHALL NOT permit `Revert` to be invoked on the SP objects of issued SPs.

Invoking `Revert` on an SP object causes the SP to revert to its Original Factory State. This method allows the TPer owner (or TPer manufacturer, if access control permits and the Maker authorities are enabled) to remove the SP owner's ownership of the SP and revert the SP to its Original Factory State.

Invocation of `Revert` is permitted on Manufactured SPs that are in any life cycle state. Successful invocation of `Revert` on a Manufactured SP that is in the Manufactured-Inactive life cycle state SHALL have no effect on the SP.

This method operates within a Read-Write session to the Admin SP. The TPer SHALL revert the SP immediately after the method is successfully invoked outside of a transaction. If `Revert` is invoked on the Admin SP's object in the `SP` table, the TPer SHALL abort the session immediately after reporting status of the method invocation if invoked outside of a transaction. The TPer MAY prepare a `CloseSession` method for retrieval by the host to indicate that the session has been aborted.

The MethodID for `Revert` SHALL be 00 00 00 06 00 00 02 02.

5.1.2.1 Revert Support

Support for `Revert` within transactions is (N), and the behavior is out of the scope of this document.

Support for `Revert` on the Admin SP's object in the `SP` table is mandatory. (Note that the OFS of the Admin SP is Manufactured, see section 5.2.2).

If the Locking SP was created in manufacturing, support for `Revert` on the Locking SP's object in the `SP` Table is mandatory.

5.1.2.2 Side effects of Revert

Upon successful invocation of the `Revert` method, the following changes SHALL be made:

- If the Locking SP is not in the Manufactured-Inactive life cycle state, then successful invocation of the `Revert` method on the Locking SP or Admin SP SHALL cause user data removal as defined by the `ActiveDataRemovalMechanism` (see Table 30).
- If the Locking SP is in the Manufactured-Inactive life cycle state, then successful invocation of the `Revert` method on the Locking SP SHALL NOT cause user data removal in the SD.

Interactions with interface commands during the processing of the `Revert` method are defined in [3].

If any TCG reset occurs prior to completing user data removal in the SD, then the operation SHALL be aborted and the Locking SP SHALL NOT revert to its Original Factory State.

Begin Informative Content

If any TCG reset occurs during the processing of the `Revert` method, the result of user data removal is undefined and the TPer does not erase personalization of the Locking SP. For example, the PIN column value for each row in `C_PIN` table is unchanged.

End Informative Content

Upon completion of user data removal in the SD, or if the Locking SP is in the Manufactured-Inactive life cycle state, the following changes SHALL be made:

- The row in the Admin SP's `SP` table that represents the invoked SP SHALL revert to its original factory values.
- The SP itself SHALL revert to its Original Factory State. While reverting to its Original Factory State, the TPer SHALL erase all personalization of the SP, and return personalized values to their Original Factory State values. The mechanism for erasure of personalization is implementation-specific.
- When `Revert` is successfully invoked on the SP object for the Admin SP (UID = 00 00 02 05 00 00 00 01), the **entire TPer** SHALL revert to its Original Factory State, including;
 - All Admin SP personalization with the exception of the PIN column value of the `C_PIN_SID` object. See section 5.1.2.2.1 for the effects of `Revert` upon the PIN column value of the `C_PIN_SID` object.
 - All issued SPs SHALL be deleted, and all Manufactured SPs SHALL revert to Original Factory State. Manufactured SPs in the Manufactured-Inactive life cycle state SHALL NOT be affected.
- Any TPer functionality affected by the life cycle state of the SP based on the templates incorporated into it is modified as defined in the appropriate Template reference section of [2], and as defined in the "State transitions for Manufactured SPs" section (section 5.2.2.2) and "State behaviors for Manufactured SPs" section (section 5.2.2.3) of this specification.

5.1.2.2.1 Effects of Revert on the PIN Column Value of `C_PIN_SID`

When `Revert` is successfully invoked on the SP object for the Admin SP (UID = 00 00 02 05 00 00 00 01), the PIN column value of the `C_PIN_SID` object SHALL be affected as follows:

1. If the SID authority has never been successfully authenticated, then the `C_PIN_SID` PIN column SHALL remain at its current value.
2. If the SID authority has previously been successfully authenticated, then the `C_PIN_SID` PIN column SHALL be set to the PIN column value of the `C_PIN_MSID` object. Additionally, the "Initial `C_PIN_SID` PIN Indicator" field SHALL be set to 0x00 upon completion of the `Revert`.

5.1.3 RevertSP – Base Template SP Method

`RevertSP` is a Pyrite SSC-specific method for managing the life cycle of an SP, if it was created in manufacturing (Manufactured SP).

```
ThisSP.RevertSP[ KeepData = boolean ]  
=>  
[ ]
```

`RevertSP` is an SP method in the Base Template.

Invoking `RevertSP` on an SP SHALL cause it to revert to its Original Factory State. This method allows the SP owner to relinquish control of the SP and revert the SP to its Original Factory State.

This method operates within a Read-Write session to an SP. The TPer SHALL revert the SP immediately after the method is successfully invoked outside of a transaction. Upon completion of reverting the SP, the TPer SHALL report status of the method invocation if invoked outside of a transaction, and then immediately abort the session. The TPer MAY prepare a `CloseSession` method for retrieval by the host to indicate that the session has been aborted.

The MethodID for `RevertSP` SHALL be 00 00 00 06 00 00 00 11.

5.1.3.1 RevertSP Support

Support for `RevertSP` within transactions is (N), and the behavior is out of the scope of this document.

If the Locking SP was created in manufacturing, support for `RevertSP` on the Locking SP is mandatory.

5.1.3.2 KeepData parameter (Locking Template-specific)

The optional **KeepData** parameter is a Locking Template-specific optional parameter. This parameter provides a mechanism for the Locking SP to be “turned off” without removing data for the Global locking range. This allows the TCG management of the SD's locking features to be disabled without causing data removal of the user data associated with the Global locking range.

When this parameter is present and set to True, the TPer SHALL NOT erase data associated with the Global locking range after the Locking SP transitions to the “Manufactured-Inactive” state even if the valid value is set to the ActiveDataRemovalMechanism parameter in DataRemovalMechanism table.

If the Global Range is either Read Unlocked or Write Unlocked at the time of invocation of `RevertSP`, then the TPer shall comply with the request to keep the user data associated with the Global locking range.

If the Global Range is Read Locked and Write Locked then invocation of the `RevertSP` method with the `KeepData` parameter set to True SHALL fail with status FAIL, and the SP SHALL NOT change life cycle states.

If the Locking SP was created in manufacturing, support for the `KeepData` parameter is mandatory for the Locking SP.

The parameter number for **KeepData** SHALL be 0x060000.

5.1.3.3 Side effects of RevertSP

Upon successful invocation of the `RevertSP` method, the following changes SHALL be made:

- If the **KeepData** parameter is not present or set to False, then successful invocation of the `RevertSP` method on the Locking SP or Admin SP SHALL cause user data removal as defined by the ActiveDataRemovalMechanism (see Table 30).
- If the **KeepData** parameter is set to True, then successful invocation of the `RevertSP` method on the Locking SP SHALL NOT cause user data removal in the SD.

Interactions with interface commands during the processing of the `RevertSP` method are defined in [3].

If any TCG reset occurs prior to completing user data removal in the SD, then the operation SHALL be aborted and the Locking SP SHALL NOT revert to its Original Factory State.

Begin Informative Content

If any TCG reset occurs during the processing of the `RevertSP` method, the result of user data removal is undefined.

End Informative Content

Upon completion of user data removal in the SD, or if the **KeepData** parameter is set to True, the following changes SHALL be made:

- The row in the Admin SP's `SP` table that represents the Locking SP SHALL revert to its original factory value.
- The Locking SP itself SHALL revert to its Original Factory State. While reverting to its Original Factory State, the TPer SHALL erase all personalization of the SP, and return the personalized values to their Original Factory State values. The mechanism for erasure of personalization is implementation-specific.
- Any TPer functionality affected by the life cycle state of the SP based on the templates incorporated into it is modified as defined in the appropriate Template reference section of [2], and as defined in the “State transitions for Manufactured SPs” section (section 5.2.2.2) and “State behaviors for Manufactured SPs” section (section 5.2.2.3) of this specification.

5.2 Life Cycle

5.2.1 Issued vs. Manufactured SPs

5.2.1.1 Issued SPs

[2] describes the life cycle states for SPs that are created through the issuance process. For Pyrite SSC-compliant TPer that support issuance, refer to [2] for the life cycle states and life cycle management.

5.2.1.2 Manufactured SPs

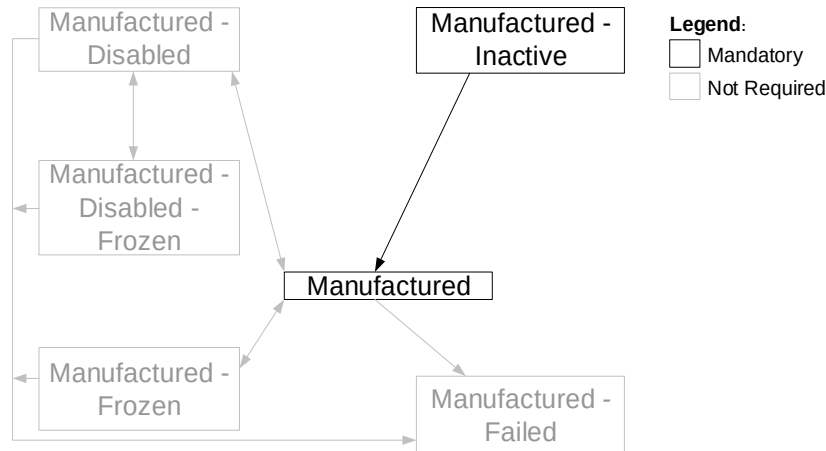
[2] defines the life cycle and life cycle management of Manufactured SPs as implementation-specific.

Pyrite SSC-compliant SPs that are created in manufacturing (Manufactured SPs) SHALL NOT have an implementation-specific life cycle, and SHALL conform to the life cycle defined in section 5.2.2.

5.2.2 Manufactured SP Life Cycle States

The state diagram for Manufactured SPs is shown in Figure 1.

Figure 1 Life Cycle State Diagram for Manufactured SPs



Additional state transitions may exist depending on the states supported by the SD and the SP's Original Factory State. Invoking `Revert` or `RevertSP` (see sections 5.1.2 and 5.1.3) on the SP will cause the SP to transition back to its Original Factory State.

The Original Factory State of the Admin SP SHALL be Manufactured. The only state that is mandatory for the Admin SP is Manufactured.

If the Locking SP is a Manufactured SP, its Original Factory State SHALL be Manufactured-Inactive. Support for Locking SP states of Manufactured and Manufactured-Inactive are mandatory.

The other states in the state diagram are beyond the scope of this document.

5.2.2.1 State definitions for Manufactured SPs

1. **Manufactured-Inactive:** This is the Original Factory State for SPs that are created in manufacturing, where it is not desirable for the functionality of that SP to be active when the TPer is shipped. All templates that exist in an SP that is in the Manufactured-Inactive state SHALL be counted in the `Instances` column of the appropriate objects in the Admin SP's `Template` table. Sessions cannot be opened to SPs in the Manufactured-Inactive state. Only SPs whose Original Factory State was Manufactured-Inactive can return to the Manufactured-Inactive state.

If the Locking SP is a Manufactured SP, support for the Manufactured-Inactive state is mandatory for the Locking SP.

2. **Manufactured:** This is the standard operational state of a Manufactured SP, and defines the initial required access control settings of an SP based on the Templates incorporated into the SP, prior to personalization.

The Manufactured state is mandatory for the Admin SP.

If the Locking SP is a Manufactured SP, support for the Manufactured state is mandatory for the Locking SP.

5.2.2.2 State transitions for Manufactured SPs

The following sections describe the mandatory and optional state transitions for Pyrite SSC-compliant Manufactured SPs.

For the Admin SP, the only transition for which support is mandatory is “ANY STATE to ORIGINAL FACTORY STATE” (section 5.2.2.2.2). As the only mandatory state for the Admin SP is Manufactured, the only mandatory transition is from Manufactured to Manufactured with the side effect of reverting the entire TPer to its Original Factory State. See section 5.1.2 for details.

If the Locking SP is a Manufactured SP, support for the “ANY STATE to ORIGINAL FACTORY STATE” transition (section 5.2.2.2.2) is mandatory. Specifically:

- Support for the transition from Manufactured to Manufactured-Inactive is mandatory. This transition is accomplished via the `Revert` or `RevertSP` method (see section 5.1.2 and section 5.1.3).
- Support for the “Manufactured-Inactive to Manufactured” transition (section 5.2.2.2.1) is mandatory. This transition is accomplished via the `Activate` method (see section 5.1).

5.2.2.2.1 Manufactured-Inactive to Manufactured

Triggers:

- The `Activate` method (see section 5.1) is successfully invoked on the SP's object in the Admin SP's `SP` table.

Side effects:

- The value in the `LifeCycleState` column of the SP's object in the Admin SP's `SP` table changes to `Manufactured`.
- The current SID PIN (`C_PIN_SID`) in the Admin SP is copied into the `PIN` column of Admin1's `C_PIN` credential (`C_PIN_Admin1`) in the activated SP. This allows for taking ownership of the SP with a known PIN credential.
- Any functionality enabled by the templates incorporated into the SP becomes active.

When the Locking SP transitions from the Manufactured-Inactive state to the Manufactured state (via invocation of the `Activate` method), the SD SHALL NOT destroy any user data.

5.2.2.2.2 ANY STATE to ORIGINAL FACTORY STATE

Triggers:

- `Revert` or `RevertSP` is successfully invoked on the SP.

Side effects:

- The value in the `LifeCycleState` column of the SP's object in the Admin SP's `SP` table changes to the value of the SP's Original Factory State.
- The SP itself reverts to its Original Factory State, as described in the section 5.1.2 and section 5.1.3.

- If the SP's Original Factory State was Manufactured-Inactive, any functionality enabled by the templates incorporated into the SP becomes inactive.

5.2.2.3 State behaviors for Manufactured SPs

5.2.2.3.1 *Manufactured-Inactive*

Any functionality enabled by the templates incorporated into the SP is inactive in this state. Sessions cannot be opened to SPs in this state.

When the Locking SP is in the Manufactured-Inactive state, the TCG management of the SD's locking features SHALL be disabled.

5.2.2.3.2 *Manufactured*

Behavior of an SP in the Manufactured state is identical to the behavior of an SP in the Issued state, as described by [2].

When the Locking SP is in the Manufactured state, the TCG management of the SD's locking features SHALL be enabled.

5.2.3 Type Table Modification

In order to accommodate the additional life cycle states defined in Pyrite, the `life_cycle_state` type SHALL be defined as follows for Pyrite:

Table 44 LifeCycle Type Table Modification

UID	Name	Format	Size	Description
00 00 00 05 00 00 04 05	life_cycle_state	Enumeration_Type, 0, 15		Used to represent the current life cycle state. The valid values are: 0 = issued, 1 = issued-disabled, 2 = issued-frozen, 3 = issued-disabled-frozen, 4 = issued-failed, 5-7 = reserved, 8 = manufactured-inactive, 9 = manufactured, 10 = manufactured-disabled, 11 = manufactured-frozen, 12 = manufactured-disabled-frozen, 13 = manufactured-failed, 14-15 = reserved

5.3 Byte Table Access Granularity

Begin Informative Content

While the general architecture defined in [2] allows data to be written into byte tables starting at any arbitrary byte boundary and with any arbitrary byte length, certain types of storage devices work more efficiently when data is written aligned to a larger block boundary. This section defines extensions to [2] that allow a device to report the restrictions that it enforces when the host invokes the `Set` method on byte tables.

End Informative Content

5.3.1 Table Table Modification

In order to allow a storage device to report its mandatory and recommended data alignment restrictions when accessing byte tables, the `Table` table SHALL contain the additional columns shown in Table 45.

Table 45 Table Table Additional Columns

Column Number	Column Name	IsUnique	Column Type
0x0D	MandatoryWriteGranularity		uinteger_4
0x0E	RecommendedAccessGranularity		uinteger_4

5.3.1.1 MandatoryWriteGranularity

This column is used to report the granularity that the storage device enforces when the host invokes the `Set` method on byte tables.

This column SHALL NOT be modifiable by the host.

5.3.1.1.1 Object Tables

For rows in the `Table` table that pertain to object tables, the value of this column SHALL be zero.

5.3.1.1.2 Byte Tables

For rows in the `Table` table that pertain to byte tables, this column indicates the mandatory access granularity (in bytes) for the `Set` method for the table described in these rows of the `Table` table. The `MandatoryWriteGranularity` column indicates the alignment requirement for both the access start offset (the `Where` parameter) and length (number of bytes in the `Values` parameter).

The value of this column SHALL be less than or equal to the value in the `RecommendedAccessGranularity` column in the same row of the `Table` table.

`MandatoryWriteGranularity` SHALL be less than or equal to 8192.

When the host invokes the `Set` method on a byte table, if `ValidMandatoryGranularity` (see Figure 2) is `False`, then the method SHALL fail with status `INVALID_PARAMETER`.

If the TPer does not have a requirement on mandatory alignment for the byte table described in a row of the `Table` table, then its `MandatoryWriteGranularity` column SHALL be set to 1.

Figure 2 ValidMandatoryGranularity

For the `Set` method:
 ValidMandatoryGranularity is True if
 a) $(x \text{ modulo } \text{MandatoryWriteGranularity}) = 0$
 and
 b) $(y \text{ modulo } \text{MandatoryWriteGranularity}) = 0$
 where:
 x = the start offset of the `Set` method
 (i.e., the value of the `Where` parameter)
 y = the number of data bytes being set
 (i.e., the length of the `Values` parameter)

5.3.1.2 RecommendedAccessGranularity

This column is used to report the granularity that the storage device recommends when the host invokes the `Set` or `Get` method on byte tables.

This column SHALL NOT be modifiable by the host.

5.3.1.2.1 Object Tables

For rows in the `Table` table that pertain to object tables, the value of this column SHALL be zero.

5.3.1.2.2 Byte Tables

For rows in the `Table` table that pertain to byte tables, this column indicates the recommended access granularity (in bytes) for the `Set` and `Get` method for the table described in these rows of the `Table` table. The `RecommendedAccessGranularity` column indicates the alignment of data for the `Set` and `Get` method that allows for optimal `Set/Get` performance.

If the TPer does not have a recommended alignment for the byte table described in a row of the `Table` table, then its `RecommendedAccessGranularity` column SHALL be set to 1.

When the host invokes the `Set` method on a byte table, if `ValidRecommendedGranularity` (see Figure 3) is False, then the performance of the TPer MAY be reduced when processing the method.

Figure 3 ValidRecommendedGranularity for Set

For the `Set` method:
 ValidRecommendedGranularity is True if
 a) $(x \text{ modulo } \text{RecommendedAccessGranularity}) = 0$
 and
 b) $(y \text{ modulo } \text{RecommendedAccessGranularity}) = 0$
 where:
 x = the start offset of the `Set` method
 (i.e., the value of the `Where` parameter)
 y = the number of data bytes being set
 (i.e., the length of the `Values` parameter)

When the host invokes the `Get` method on a byte table, if `ValidRecommendedGranularity` (see Figure 4) is `False`, then the performance of the TPer MAY be reduced when processing the method.

Figure 4 ValidRecommendedGranularity for Get

For the `Get` method:

`ValidRecommendedGranularity` is `True` if

a) $(x \text{ modulo } \text{RecommendedAccessGranularity}) = 0$

and

b) $(y \text{ modulo } \text{RecommendedAccessGranularity}) = 0$

where:

x = the start offset of the `Get` method

(i.e., the value of the `startRow` component of the `Cellblock` parameter)

y = the number of data bytes being retrieved

(i.e., the difference of the `endRow` and `startRow` components of the `Cellblock` parameter, plus one)